

Mem0：分层记忆 + 三路召回的长期记忆方案 已付费

原创 神说要有光 神光的幸福生活 2026年6月6日 20:15 山东 标题已修改

我们用 DeepAgents 的中间件做了消息摘要和截断，并用 Redis 搭建短期记忆存储。

而长期记忆要放到向量数据库 中，通过语义检索

但只是语义检索还不够，最好还要支持关键词检索、知识图谱检索

- 语义检索：基于向量数据库实现语义相似度匹配，支持模糊意图、同义表述的历史记忆召回；
- 关键词检索：构建倒排索引，针对实体、专有名词、关键参数等内容做精准检索补充；
- 知识图谱推理检索：通过抽取实体与实体间关系，实现多跳关联检索与逻辑推理，强化记忆的关联挖掘能力。

三类检索能力协同工作，三路融合召回，以提升长期记忆召回的准确率与有效性。

但自己做这个还是挺麻烦的，有没有啥现成的长期记忆方案呢？

有的，就是 mem0

它是业界主流的 Agent 长期记忆库，原生集成向量语义检索、BM25 关键词检索、知识图谱关联检索三大能力，默认完成三路检索的融合打分与结果排序，完全契合我们的技术需求。

引入 Mem0 后，无需从零实现记忆检索能力，能够大幅缩短项目落地周期，降低开发成本，快速上线具备完整三路召回能力的长期记忆体系。

我们来用一下：

```
mkdir mem0-test  
cd mem0-test  
npm init -y
```

安装依赖：

```
pnpm install mem0ai dotenv
```

创建 src/mem0-test.mjs

```

import 'dotenv/config';
import { MemoryClient } from 'mem0ai';

const USER_ID = 'demo-user';

function log(title, data) {
  console.log(`\n=== ${title} ===`);
  console.log(typeof data === 'string' ? data : JSON.stringify(data, null, 2));
}

asyncfunction main() {

  const client = new MemoryClient({ apiKey: process.env.MEM0_API_KEY });

  const conversation = [
    { role: 'user', content: '我是素食主义者，而且对坚果过敏。' },
    { role: 'assistant', content: '好的，我会记住你的饮食偏好。' },
    { role: 'user', content: '我住在北京，平时喜欢跑步。' },
    { role: 'assistant', content: '已记录：北京、爱好跑步。' },
  ];

  const added = await client.add(conversation, { userId: USER_ID });
  log('添加记忆', added);

  const searchResult = await client.search('用户的饮食限制是什么？中文回答', {
    filters: { user_id: USER_ID },
    topK: 5,
  });
  log('搜索记忆', searchResult);

  const allMemories = await client.getAll({
    filters: { user_id: USER_ID },
    pageSize: 10,
  });
  log('列出全部记忆', allMemories);

  const firstMemory = allMemories.results?.[0] ?? searchResult.results?.[0];
  if (firstMemory?.id) {
    const memory = await client.get(firstMemory.id);
    log('获取单条记忆', memory);

    const updated = await client.update(firstMemory.id, {
      text: `${memory.memory} ?? firstMemory.memory` (已通过示例脚本更新) `，
    });
    log('更新记忆', updated);

    const history = await client.history(firstMemory.id);
  }
}

```

```

        log('记忆变更历史', history);
    }

    if (process.argv.includes('--cleanup')) {
        const deleted = await client.deleteAll({ userId: USER_ID });
        log('清理测试数据', deleted);
    } else {
        console.log('\n提示: 运行 `node src/mem0-test.mjs --cleanup` 可删除本次测试用户的全部记忆')
    }
}

main().catch((error) => {
    console.error('\n执行失败:', error.message ?? error);
    if (error.suggestion) {
        console.error('建议:', error.suggestion);
    }
    process.exit(1);
});

```

跑一下:

神光的编程秘籍

mem0 只要配一个 apikey 就可以存储记忆、检索记忆了。

但免费版是有调用次数限制的:

神光的编程秘籍

后面我们自己部署一下。

mem0 一共有三种记忆的作用范围：

用户记忆 (User Memory)

绑定 userId，存的是跟着人走的长期信息：姓名、城市、偏好、习惯等。

换会话、换 Agent 也能读到，适合个性化和跨场景复用。

会话记忆 (Session Memory)

绑定 `userId + runId`，存的是当前这次对话里的任务上下文：这次要做什么、讨论到哪一步、临时目标等。

会话结束后可以清理，不会和别的 `thread` 混在一起。

Agent 记忆 (Agent Memory)

绑定 `agentId`，存的是某个 Agent 自己的设定：角色、语气、回答方式、专业领域等。

同一个用户换不同 Agent，各自保持独立人格和风格。

用户记忆管“这个人”，会话记忆管“这次聊什么”，Agent 记忆管“这个助手是谁”。

我们分别存取一下试试：

`src/mem0-scoped-memory-test.mjs`

```
/**
 * Mem0 三种记忆 scope 的 API 测试
 *
 * add / search 分开调用，自行决定何时 search (add 为异步处理)
 *
 */
import "dotenv/config";
import { MemoryClient } from "mem0ai";

const USER_ID = "mem0_test_user";
const RUN_ID = "mem0_test_session";
const AGENT_ID = "mem0_test_agent";

function log(title, data) {
  console.log(`\n=== ${title} ===`);
  console.log(typeof data === "string" ? data : JSON.stringify(data, null, 2));
}

asyncfunction addUserMemory(client) {
  const messages = [
    { role: "user", content: "我叫小明，住在杭州，平时喜欢骑行和摄影。" },
    { role: "assistant", content: "好的，已记住你的姓名、城市和爱好。" },
  ];
  const added = await client.add(messages, { userId: USER_ID });
  log("用户记忆 - add", added);
}

asyncfunction searchUserMemory(client) {
```

```

const searched = await client.search("用户住在哪里, 有什么爱好", {
  filters: { user_id: USER_ID },
  topK: 5,
});
log("用户记忆 - search", searched.results?.map((m) => m.memory) ?? []);

const listed = await client.getAll({ filters: { user_id: USER_ID }, pageSize: 5 });
log("用户记忆 - getAll", listed.results?.map((m) => m.memory) ?? []);
}

asyncfunction addSessionMemory(client) {
const messages = [
  { role: "user", content: "这次聊天先帮我把季度总结的大纲列出来, 重点写 Q1 的项目复盘。" },
  { role: "assistant", content: "明白, 我们先围绕 Q1 项目复盘整理季度总结大纲。" },
];
const added = await client.add(messages, { userId: USER_ID, runId: RUN_ID });
log("会话记忆 - add", added);
}

asyncfunction searchSessionMemory(client) {
const searched = await client.search("这次对话要先做什么", {
  filters: { AND: [{ user_id: USER_ID }, { run_id: RUN_ID }] },
  topK: 5,
});
log("会话记忆 - search", searched.results?.map((m) => m.memory) ?? []);

const listed = await client.getAll({
  filters: { AND: [{ user_id: USER_ID }, { run_id: RUN_ID }] },
  pageSize: 5,
});
log("会话记忆 - getAll", listed.results?.map((m) => m.memory) ?? []);
}

asyncfunction addAgentMemory(client) {
const messages = [
  { role: "user", content: "你现在是旅行规划助手, 回答时多给具体建议和备选方案。" },
  { role: "assistant", content: "好的, 我会以旅行规划助手的身份, 提供具体建议和备选方案。" },
];
const added = await client.add(messages, { agentId: AGENT_ID });
log("Agent 记忆 - add", added);
}

asyncfunction searchAgentMemory(client) {
const searched = await client.search("这个 Agent 的角色和回答方式", {
  filters: { agent_id: AGENT_ID },
  topK: 5,
});

```

```

    log("Agent 记忆 - search", searched.results?.map((m) => m.memory) ?? []);

    const listed = await client.getAll({ filters: { agent_id: AGENT_ID }, pageSize: 5 });
    log("Agent 记忆 - getAll", listed.results?.map((m) => m.memory) ?? []);
  }

  asyncfunction main() {
    if (!process.env.MEM0_API_KEY) {
      console.error("缺少 MEM0_API_KEY");
      process.exit(1);
    }

    const client = new MemoryClient({ apiKey: process.env.MEM0_API_KEY });
    const action = process.argv[2] ?? "add";

    if (process.argv.includes("--cleanup")) {
      await client.deleteAll({ userId: USER_ID });
      await client.deleteAll({ userId: USER_ID, runId: RUN_ID });
      await client.deleteAll({ agentId: AGENT_ID });
      log("清理完成", { USER_ID, RUN_ID, AGENT_ID });
      return;
    }

    if (action === "add") {
      await addUserMemory(client);
      await addSessionMemory(client);
      await addAgentMemory(client);
      console.log("\nadd 已提交 (异步处理), 稍后再运行: pnpm scoped-memory search");
      return;
    }

    if (action === "search") {
      await searchUserMemory(client);
      await searchSessionMemory(client);
      await searchAgentMemory(client);
      return;
    }

    console.error(`未知命令: ${action}, 可用: add | search | --cleanup`);
    process.exit(1);
  }

  main().catch((error) => {
    console.error("\n执行失败:", error.message ?? error);
    if (error.suggestion) console.error("建议:", error.suggestion);
    process.exit(1);
  });

```


跑一下：

神光的编程秘籍

用户、会话、Agent 三种级别的 scope 可以独立的存储和检索记忆。

然后我们在 Agent 里接入 mem0 做长期记忆，用 Redis 存做短期记忆

安装依赖：

```
pnpm install @langchain/core @langchain/openai dotenv ioredis langchain mem0ai zod
```

改下 .env

```
MEM0_API_KEY=m0-xx

OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
MODEL_NAME=qwen-plus
OPENAI_API_KEY=sk-xx

# 用于身份验证，实现链路上报
LANGCHAIN_API_KEY=xx
# 指定LangSmith中的项目，追踪结果会归类到该项目下
LANGCHAIN_PROJECT=mem0-test
# 开启LangSmith追踪功能
LANGCHAIN_TRACING_V2=true

# Redis (默认与 docker-compose 一致)
```

```
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_DB=0
MEMORY_TTL_SECONDS=1800
MEMORY_SESSION_ID=session_001

# Mem0: MEM0_USER_ID=用户层, MEMORY_SESSION_ID=会话层 run_id
MEM0_USER_ID=demo_user_001
MEM0_TOP_K=5
```

创建 docker-compose.yml （我们要跑 redis 来存短期记忆）

```

services:
  # Redis
  redis:
    image:redis:7-alpine
    container_name:redis-mem0-test
    restart:always
    ports:
      -"6379:6379"
    volumes:
      -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/redis:/data
    command:redis-server--appendonlyyes
    healthcheck:
      test:["CMD","redis-cli","ping"]
      interval:5s
      timeout:5s
      retries:5

# Redis 官方 Web GUI (类似 pgAdmin)
redisinsight:
  image:redis/redisinsight:2.50
  container_name:redisinsight-mem0-test
  restart:always
  ports:
    -"5540:5540"
  volumes:
    -${DOCKER_VOLUME_DIRECTORY:-.}/volumes/redisinsight:/data
  environment:
    -RI_HOST=0.0.0.0
  depends_on:
    -redis

networks:
  default:
    name:common-network

```

创建 mem0-redis-mem0-agent.mjs

```

/**
 * Redis 记这轮聊天, Mem0 记值得长期留着的事。
 *
 * 用户层 = 换天聊还认得你; 会话层 = 只管当前这个聊天窗口。
 *
 * docker compose up -d redis 后 pnpm agent
 * :clear 清 Redis | :clear-mem0 清 Mem0 | exit/:q 退出
 */

```

```

import "dotenv/config";
import Redis from "ioredis";
import * as readline from "node:readline/promises";
import { stdin, stdout } from "node:process";
import { z } from "zod";
import { MemoryClient } from "mem0ai";
import { ChatOpenAI } from "@langchain/openai";
import {
  SystemMessage,
  SystemMessageChunk,
  HumanMessage,
  mapChatMessagesToStoredMessages,
  mapStoredMessagesToChatMessages,
} from "@langchain/core/messages";
import { createAgent, summarizationMiddleware } from "langchain";

const REDIS_HOST = process.env.REDIS_HOST ?? "localhost";
const REDIS_PORT = Number(process.env.REDIS_PORT ?? 6379);
const REDIS_DB = Number(process.env.REDIS_DB ?? 0);
const MEMORY_TTL = Number(process.env.MEMORY_TTL_SECONDS ?? 1800);
const KEY_PREFIX = process.env.MEMORY_KEY_PREFIX ?? "agent:short_memory";

const USER_ID = process.env.MEM0_USER_ID ?? "demo_user_001";
const SESSION_ID = process.env.MEMORY_SESSION_ID ?? "session_001";
const MEM0_TOP_K = Number(process.env.MEM0_TOP_K ?? 5);

const memorySchema = z.object({
  write_user: z
    .boolean()
    .describe(
      "写入用户层：换一个新会话仍应保留的长期事实（身份、居住地、长期爱好、饮食禁忌、持久偏好）。不含仅本轮",
    ),
  write_session: z
    .boolean()
    .describe(
      "写入会话层：仅当前会话/thread 有效的任务、大纲、进度、待办、临时决策（如「这次先写...」「数据部分明",
    ),
  reason: z.string().describe("分类理由，一句话"),
});

const CLASSIFIER_PROMPT = `你是记忆分层分类器。判断本轮对话是否有「新事实」需写入 Mem0，并分到正确层！

## user 层（跨会话长期）
- 用户身份与画像：姓名、职业、居住地、长期爱好
- 长期偏好与约束：饮食过敏、回答风格、常用技术栈
- 持续数周以上的个人背景（非单次任务）

```

session 层（仅当前会话）

- 当前正在做的任务、目标、文档大纲、方案草稿
- 本会话内的进度、决策、待办、临时约定
- 用户明确用「这次」「本轮」「当前会话」描述的工作上下文

均不写入

- 寒暄、致谢、纯确认
- 助手生成的通用内容（攻略、示例代码、建议清单），用户未明确采纳为新事实
- 无信息增量的复述

决策原则

1. 「这次我们先写 Q1 总结」「当前在排查 XX」→ 优先 session，不要标成 user
2. user 与 session 可同时为 true（如同时说职业+当前任务），但勿把纯会话任务只标 user
3. 一次性请求（如「帮我做旅行攻略」）且未产生需跨轮记住的约定 → 均为 false`；

const summaryPrompt = `你是对话摘要助手。用中文简洁总结：话题、会话内进度/报错/待办。

用户级长期偏好由外部记忆维护，摘要勿重复堆砌。不要编造。

待摘要的对话：

{messages}

摘要：`；

/** Mem0 注入的 SystemMessage 不写回 Redis */

function messagesForRedis(messages) {

return messages.filter(

(m) => !SystemMessage.isInstance(m) && !SystemMessageChunk.isInstance(m),

);

}

class RedisMessageStore {

constructor({ redis, keyPrefix, ttlSeconds }) {

this.redis = redis;

this.keyPrefix = keyPrefix;

this.ttlSeconds = ttlSeconds;

}

messagesKey(sessionId) {

return `\${this.keyPrefix}:\${sessionId}:messages`;

}

async loadMessages(sessionId) {

const raw = await this.redis.get(this.messagesKey(sessionId));

if (!raw) return [];

return mapStoredMessagesToChatMessages(JSON.parse(raw));

}

```

async saveMessages(sessionId, messages) {
  const payload = JSON.stringify(mapChatMessagesToStoredMessages(messages));
  await this.redis.set(this.messagesKey(sessionId), payload, "EX", this.ttlSeconds);
}

async clear(sessionId) {
  await this.redis.del(this.messagesKey(sessionId));
}

async ttl(sessionId) {
  return this.redis.ttl(this.messagesKey(sessionId));
}
}

class MemMemoryStore {
  constructor({ client, userId, sessionId, topK, classifier }) {
    this.client = client;
    this.userId = userId;
    this.sessionId = sessionId;
    this.topK = topK;
    this.classifier = classifier;
  }

  async search(query) {
    const [userRes, sessionRes] = await Promise.all([
      this.client.search(query, {
        filters: { user_id: this.userId },
        topK: this.topK,
      }),
      this.client.search(query, {
        filters: {
          AND: [{ user_id: this.userId }, { run_id: this.sessionId }],
        },
        topK: this.topK,
      }),
    ]);
    return {
      user: userRes.results ?? [],
      session: sessionRes.results ?? [],
    };
  }
}

buildSystemMessage({ user, session }) {
  const blocks = [];
  if (user.length) {
    blocks.push(`【用户长期记忆】\n${user.map((m) => `- ${m.memory}`)}.join("\n")}`);
  }
  if (session.length) {

```

```

        blocks.push(`【当前会话记忆】\n${session.map((m) => `- ${m.memory}`).join("\n")}`);
    }
    if (!blocks.length) return null;
    return new SystemMessage(`${blocks.join("\n\n")}\n\n请结合以上记忆回答，勿编造。`);
}

async classifyAndPersist(userText, assistantText) {
    const turn = [
        { role: "user", content: userText },
        { role: "assistant", content: assistantText },
    ];

    const { write_user, write_session, reason } = await this.classifier.invoke([
        new SystemMessage(CLASSIFIER_PROMPT),
        new HumanMessage(`用户: ${userText}\n助手: ${assistantText}`),
    ]);

    const written = [];
    if (write_user) {
        await this.client.add(turn, { userId: this.userId });
        written.push("user");
    }
    if (write_session) {
        await this.client.add(turn, { userId: this.userId, runId: this.sessionId });
        written.push("session");
    }
    return { written, reason };
}

async clear() {
    await this.client.deleteAll({ userId: this.userId });
    await this.client.deleteAll({ userId: this.userId, runId: this.sessionId });
}

async function invokeWithMemory(agent, redisStore, mem0Store, sessionId, userText) {
    const history = await redisStore.loadMessages(sessionId);
    console.log(` ↳ Redis 加载 ${history.length} 条历史`);

    const mem = await mem0Store.search(userText);
    if (mem.user.length) console.log(` ↳ Mem0 用户层 ${mem.user.length} 条`);
    if (mem.session.length) console.log(` ↳ Mem0 会话层 ${mem.session.length} 条`);

    const memoryMsg = mem0Store.buildSystemMessage(mem);
    const invokeMessages = [
        ...(memoryMsg ? [memoryMsg] : []),
        ...history,
        new HumanMessage(userText),
    ];
}

```

```

];

const result = await agent.invoke(
  { messages: invokeMessages },
  { recursionLimit: 30 },
);

const redisMessages = messagesForRedis(result.messages);
const dropped = result.messages.length - redisMessages.length;
await redisStore.saveMessages(sessionId, redisMessages);
const ttl = await redisStore.ttl(sessionId);
console.log(
  `  ↳ Redis 写回 ${redisMessages.length} 条` +
    (dropped ? ` (过滤 ${dropped} 条 SystemMessage)` : "") +
    ` (TTL ${ttl}s)`,
);

const assistantText = String(result.messages.at(-1)?.content ?? "");
const { written, reason } = await mem0Store.classifyAndPersist(userText, assistantText);
console.log(`  ↳ 分类: ${reason}`);
console.log(written.length ? `  ↳ Mem0 写入: ${written.join(", ")}` : "  ↳ Mem0 未写入");

return { messages: result.messages, redisMessages, assistantText };
}

if (!process.env.MEM0_API_KEY || !process.env.OPENAI_API_KEY) {
  console.error("需要 MEM0_API_KEY 与 OPENAI_API_KEY");
  process.exit(1);
}

const redis = new Redis({ host: REDIS_HOST, port: REDIS_PORT, db: REDIS_DB });
const mem0 = new MemoryClient({ apiKey: process.env.MEM0_API_KEY });

redis.on("connect", () => console.log("✅ Redis 已连接"));
redis.on("error", (err) => console.error("❌ Redis 错误:", err.message));

try {
  await redis.ping();
} catch {
  console.error("Redis 未连接, 请先执行: docker compose up -d redis");
  process.exit(1);
}

const redisStore = new RedisMessageStore({
  redis,
  keyPrefix: KEY_PREFIX,
  ttlSeconds: MEMORY_TTL,

```



```

});

const llmOpts = {
  apiKey: process.env.OPENAI_API_KEY,
  configuration: { baseUrl: process.env.OPENAI_BASE_URL },
  temperature: 0,
};

const model = new ChatOpenAI({ model: process.env.MODEL_NAME, ...llmOpts });

const classifier = new ChatOpenAI({
  model: process.env.MODEL_NAME,
  ...llmOpts,
}).withStructuredOutput(memorySchema);

const mem0Store = new Mem0MemoryStore({
  client: mem0,
  userId: USER_ID,
  sessionId: SESSION_ID,
  topK: MEM0_TOP_K,
  classifier,
});

const agent = createAgent({
  model,
  tools: [],
  systemPrompt:
    "你是会话助手。结合系统消息中的长期/会话记忆回答，中文简短。有对话摘要则据此继续。",
  middleware: [
    summarizationMiddleware({
      model,
      summaryPrompt,
      trigger: { messages: 8 },
      keep: { messages: 4 },
    }),
  ],
});

console.log(`用户 ${USER_ID} | 会话 ${SESSION_ID}`);
console.log("输入 exit / quit / :q 退出; :clear 清空 Redis; :clear-mem0 清空 Mem0\n");

const rl = readline.createInterface({ input: stdin, output: stdout });
let prevCount = (await redisStore.loadMessages(SESSION_ID)).length;

try {
  while (true) {
    const userText = (await rl.question("你: ")).trim();
    if (!userText) continue;

```

```

    if (["exit", "quit", ":q"].includes(userText.toLowerCase())) break;

    if (userText === ":clear") {
        await redisStore.clear(SESSION_ID);
        prevCount = 0;
        console.log("已清空 Redis 短期记忆\n");
        continue;
    }

    if (userText === ":clear-mem0") {
        await mem0Store.clear();
        console.log("已清空 Mem0 用户层与当前会话层\n");
        continue;
    }

    const { redisMessages, assistantText } = await invokeWithMemory(
        agent,
        redisStore,
        mem0Store,
        SESSION_ID,
        userText,
    );

    console.log("\n助手:", assistantText);
    console.log(`Redis 消息数: ${redisMessages.length}`);
    if (redisMessages.length < prevCount + 2) {
        console.log("  < 已触发压缩");
    }
    prevCount = redisMessages.length;
    console.log();
} finally {
    rl.close();
}

await redis.quit();

/*
 * 测试对话（复制进终端，先来 :clear-mem0 和 :clear）
 *
 * 一、寒暄
 * 你好 / 在吗 / 谢谢
 * → 纯客套，Mem0 不用记。
 *
 * 二、自我介绍
 * 我叫小明，住在杭州，平时喜欢骑行和摄影。
 * 我对海鲜过敏，出差尽量别安排沿海城市。

```

* → 换天聊还得知道的事，写 `user` 层。

*

* 三、这会儿在干嘛

* 这次我们先写 Q1 季度总结，大纲分三块：项目复盘、数据指标、下季度计划。

* 项目复盘里重点写 `order-service` 的 500 错误排查过程。

* → 只管这次聊天的事，写 `session` 层。

*

* 四、长期背景 + 手头活

* 我长期做后端开发，这次会话的任务是排查 `payment-api` 超时，先从 P99 日志看起。

* 另外我之后技术回答都希望带代码示例，这个一直记住。

* → 职业和当前任务可能两层都写，偏好那条走 `user`。

*

* 五、Redis 和 Mem0 各管啥

* 刚才说的 `payment-api`，超时阈值先假设 3 秒。

* 上一句我说的阈值是多少？

* → 刚说过的话 Redis 兜得住，不用等 Mem0。

*

* 重启 agent (别清 mem0) 再问：我是谁？有什么过敏？

* → 新会话 Redis 是空的，`user` 层还能认出你。

*

* 六、聊多了会压缩 (可选，连聊 8 轮以上)

* 继续完善 Q1 总结 / 把第二段改短 / 加个标题.....

* → 终端会出现「已触发压缩」，老消息变摘要。

*

* 推荐顺序：清空 → 寒暄 → 自我介绍 → 当前任务 → 重启验 `user` → 清 mem0 验 `session` 没了

*/

跑一下：

这样，我们基于 Redis 实现了短期记忆，基于 Mem0 实现了长期记忆

豆包那种跨会话还能记住用户信息，就是做了长期记忆分层

用户相关的写入用户层，可以跨会话检索

当前会话相关的写入会话层，只在当前会话生效

每轮对话结束，调用大模型判断是 user、session 哪一层，分别存到不同层的长期记忆

这样跨会话聊天可以检索用户层的记忆，用户画像、偏好相关的信息可以长期保存

但现在直接用 mem0 现成的服务有一堆问题，比如请求次数限制，记忆都是英文的等。

接下来我们自己部署一下：

下载 mem0 源码：

```
git clone git@github.com:mem0ai/mem0.git
```

它 server 目录下有 docker-compose.yaml 配置，直接跑就行。

不过要先改一下 .env 配置文件

```
OPENAI_API_KEY=sk-xx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1

# Postgres - POSTGRES_PASSWORD is required; docker-compose will not start without it.
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DB=postgres
```

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=12345678
POSTGRES_COLLECTION_NAME=memories

ADMIN_API_KEY=
JWT_SECRET=+BpwM3kDG0uzDAt32gl44PBjmuNMLcw9ToNx3V0Ivjad4G8gCfYqotABzic11zjZ
# Local development only. Leave false in production.
AUTH_DISABLED=true
DASHBOARD_URL=http://localhost:3000
APP_DB_NAME=mem0_app

# Default LLM and embedder models. Provider must be in BUNDLED_*_PROVIDERS
# (see server/main.py). Override to pin a specific model without editing code.
MEM0_DEFAULT_LLM_MODEL=qwen-plus
MEM0_DEFAULT_EMBEDDER_MODEL=text-embedding-v3

# Anonymous telemetry. Sends a single onboarding event per install
# (install UUID, email domain, server version, source). No PII.
# Set to false to opt out.
MEM0_TELEMETRY=true

# Days of request_logs history to keep when `make prune-logs` runs.
# Wire the prune command into cron/systemd in production.
REQUEST_LOG_RETENTION_DAYS=30
```

然后还要改几个地方：

```
"packageManager": "pnpm@10.5.2+sha512.da9dc28cd3ff40d0592188235ab25d3202add8a207afbedc682  
"pnpm": {  
  "onlyBuiltDependencies": [  
    "sharp",  
    "unrs-resolver"  
  ]  
}
```

跑一下：

跑起来可以看到接口文档和 mem0 的管理界面：

跑起来之后创建 api key 然后我们在代码里用连接一下：

创建 `src/mem0-local-pai-demo.mjs`


```

import "dotenv/config";

const BASE_URL = "http://localhost:8888"
const USER_ID = "local_api_demo";
const API_KEY = process.env.MEM0_LOCAL_API_KEY;

function log(title, data) {
  console.log(`\n=== ${title} ===`);
  console.log(typeof data === "string" ? data : JSON.stringify(data, null, 2));
}

class LocalMem0Client {
  constructor({ baseUrl = BASE_URL, apiKey = API_KEY } = {}) {
    this.baseUrl = baseUrl.replace(/\/$/, "");
    this.apiKey = apiKey;
  }

  headers() {
    const h = { "Content-Type": "application/json" };
    if (this.apiKey) h["X-API-Key"] = this.apiKey;
    return h;
  }

  async request(path, options = {}) {
    const res = await fetch(`${this.baseUrl}${path}`, {
      ...options,
      headers: { ...this.headers(), ...options.headers },
    });
    const text = await res.text();
    let body;
    try {
      body = text ? JSON.parse(text) : null;
    } catch {
      body = text;
    }
    if (!res.ok) {
      const detail = typeof body === "object" ? body.detail ?? JSON.stringify(body) : body;
      throw new Error(`${res.status} ${detail}`);
    }
    return body;
  }

  async add(messages, { userId, runId, agentId, metadata, infer } = {}) {
    const payload = {
      messages: typeof messages === "string"
        ? [{ role: "user", content: messages }]
        : messages,
    };
  }

```

```

        user_id: userId,
        run_id: runId,
        agent_id: agentId,
        metadata,
        infer,
    };
    returnthis.request("/memories", {
        method: "POST",
        body: JSON.stringify(payload),
    });
}

```

```

async getAll({ filters, userId, runId, agentId } = {}) {
    const params = new URLSearchParams();
    const uid = userId ?? filters?.user_id;
    const rid = runId ?? filters?.run_id;
    const aid = agentId ?? filters?.agent_id;
    if (uid) params.set("user_id", uid);
    if (rid) params.set("run_id", rid);
    if (aid) params.set("agent_id", aid);
    const qs = params.toString();
    returnthis.request(`/memories${qs} ? `?${qs}` : ""`);
}

```

```

async search(query, { filters, topK = 5 } = {}) {
    returnthis.request("/search", {
        method: "POST",
        body: JSON.stringify({ query, filters, top_k: topK }),
    });
}

```

```

async deleteAll({ userId, runId, agentId } = {}) {
    const params = new URLSearchParams();
    if (userId) params.set("user_id", userId);
    if (runId) params.set("run_id", runId);
    if (agentId) params.set("agent_id", agentId);
    returnthis.request(`/memories?${params}`, { method: "DELETE" });
}
}

```

```

asyncfunction main() {
    const client = new LocalMemClient();
    const action = process.argv[2] ?? "add";

    if (process.argv.includes("--cleanup")) {
        log("清理测试数据", await client.deleteAll({ userId: USER_ID }));
        return;
    }
}

```

```

    }

    if (action === "add") {
      const added = await client.add(
        [
          { role: "user", content: "我是素食主义者，而且对坚果过敏。" },
          { role: "assistant", content: "好的，我会记住你的饮食偏好。" },
          { role: "user", content: "我住在北京，平时喜欢跑步。" },
          { role: "assistant", content: "已记录：北京、爱好跑步。" },
        ],
        { userId: USER_ID },
      );
      log("添加记忆", added);
      return;
    }

    if (action === "search") {
      log(
        "搜索记忆",
        await client.search("用户的饮食限制是什么?", {
          filters: { user_id: USER_ID },
          topK: Number(process.env.MEM0_TOP_K ?? 5),
        }),
      );
      return;
    }

    if (action === "list") {
      log("列出全部记忆", await client.getAll({ filters: { user_id: USER_ID } }));
      return;
    }

    console.error(`未知命令: ${action}, 可用: add | search | list | --cleanup`);
    process.exit(1);
  }

  main().catch((error) => {
    console.error("\n执行失败:", error.message ?? error);
    process.exit(1);
  });
}

```

mem0 的 server 提供了 restful 的接口，用 fetch 来访问。

跑一下：

神光的编程秘籍

这样我们就基于本地跑的 mem0 服务实现了长期记忆的分层存储、语义检索。

有的同学可能怀疑，这是不是走的向量 + 关键词 + 图谱三路检索啊

你不都拿到源码了么？让 AI 分析下就好了：

确实是三路召回 + 重排。

你用 mem0 平台的 api 的时候，也能看到三路分数：

这样，我们就可以基于 mem0 来存储长期记忆了，用户层、会话层的区分，以及向量、关键词、图谱的检索，都可以直接用。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

Mem0 是开箱即用的长期记忆方案。

它分了用户层、会话层、Agent 层三种记忆。

类似豆包那种切换会话，它依然知道你的信息，就是提取用户画像等内容存在了用户层记忆。

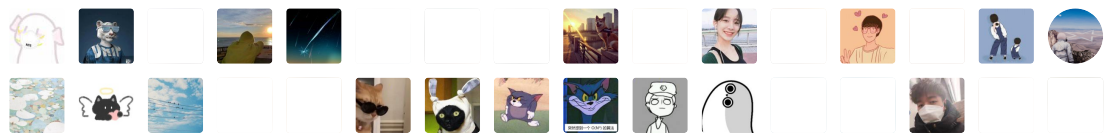
它还做了向量的语义检索、BM25 的关键词检索、图谱的推理式检索三种混合召回 + 重排。

我们可以直接用它的云端服务，但免费版有次数限制，也可以自己部署开源版本。

Mem0 是比较流行的长期记忆方案，你的 Agent 可以直接接入它来实现长期记忆存储。

我们基于 Redis 做了短期记忆存储，用 Mem0 实现了长期记忆分层存储，这样 Agent 的记忆系统就比较完善了。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

Redis：实现 Agent 短期记忆存储的最佳方案

下一篇

Nest 进阶：企业级 Node.js 后端最主流框架

修改于2026年6月7日

留言 8

写留言



冬煦 河北 5天前

有的大模型，不给返回按格式化输出output方式。要在CLASSIFIER_PROMPT提示词模板加一句“

你需要输出一个 json 对象，包含以下三个字段：

- write_user: boolean 类型，表示是否写入用户层长期记忆
 - write_session: boolean 类型，表示是否写入会话层临时记忆
 - reason: string 类型，一句话说明分类理由
- ”



神光的幸福生活 作者 5天前

大模型不行，一般用了 structured output 都可以输出 json

2



一步≡¹⁰⁶ 浙江 3天前

有了mem0就不需要向量模型把文本向量化了吗



神光的幸福生活 作者 3天前

需要啊，你没看到配置了嵌入模型，向量数据库么



神光的幸福生活 作者 3天前

mem0内部封装了



莫奈 重庆 6天前

理论和工具是不是已经学完了呀？后面的课程都是实战吗



神光的幸福生活 作者 6天前

差不多，还有面试题



天福 广东 6天前

MemoMemoryStore里的search方法发promise.all有个要传sessionId

[首评](#)